배포 및 서빙 시스템 분석

1. 배포 및 서빙 개요

Open R1의 배포 및 서빙 시스템은 대규모 클러스터 환경에서 모델을 효율적으로 서빙하고 훈련하기 위한 인프라를 제공합니다. SGLang과 vLLM을 기반으로 한 고성능 추론 서버와 Slurm을 통한 클러스터 관리를 지원합니다.

1.1 주요 구성 요소

- **SGLang 서버**: 고성능 추론 서빙
- **vLLM 백엔드**: 효율적인 추론 엔진
- **라우터 시스템**: 부하 분산 및 요청 관리
- **Slurm 통합**: 클러스터 작업 관리
- **멀티노드 지원**: 대형 모델 분산 서빙

1.2 시스템 아키텍처

```
클라이언트 요청 → 라우터 → 서버 클러스터 → 모델 추론 → 응답 반환
↓
부하 분산 및 큐 관리
```

2. SGLang 서버 배포 (`slurm/serve_r1.slurm`)

2.1 서버 설정 및 초기화

```bash
#!/bin/bash

#SBATCH --job-name=r1-server

#SBATCH --partition=hopper-prod

#SBATCH --qos=normal

#SBATCH --nodes=2 # 2노드 사용

#SBATCH --gpus-per-node=8 # 노드당 8 GPU

#SBATCH --exclusive # 전용 노드 사용

#SBATCH --time=7-00:00:00 # 7일 실행



# 환경 변수 설정

MODEL_PATH="deepseek-ai/DeepSeek-R1"

CONDA_ENV="sglang124"

SERVER_PORT=39877

DIST_PORT=45000



# 클러스터 특화 설정

export  OUTLINES_CACHE_DIR=/scratch/serve_r1/ocache/

export  TRITON_HOME=/scratch/serve_r1/triton/

export  GLOO_SOCKET_IFNAME="enp71s0"

export  NCCL_SOCKET_IFNAME="enp71s0"
```

2.2 멀티노드 서버 시작

```
# 첫 번째 노드 IP 획득
FIRST_NODE=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n1)
FIRST_NODE_IP=$(srun --nodes=1 --ntasks=1 -w "$FIRST_NODE" hostname --ip-a

# 모든 노드에서 동기적으로 서버 시작
srun --nodes=2 --ntasks=2 --ntasks-per-node=1 \
bash -c "python -m sglang.launch_server \
--model-path '$MODEL_PATH' \
--tp 16 \ # 16-way 텐서 병렬화
--dist-init-addr '$FIRST_NODE_IP:$DIST_PORT' \
--nnodes 2 \ # 2개 노드
--node-rank \$SLURM_PROCID \ # 노드 순위 자동 할당
--port '$SERVER_PORT' \
--host 0.0.0.0 \
--trust-remote-code \
--max-running-requests 56 \ # 동시 요청 제한
--context-length 32768" & # 32K 컨텍스트 길이
```

2.3 서버 상태 확인 및 헬스 체크

```
# 서버 시작 대기 (타임아웃 포함)
TIMEOUT=3600 # 1시간 타임아웃
```

```
START_TIME=$(date +%s)
echo "Waiting for SGLang server (http://$FIRST_NODE_IP:$SERVER_PORT)..."

while true; do
if curl -s -o /dev/null -w "%{http_code}" "http://$FIRST_NODE_IP:$SERVER_
echo "Server is ready at http://$FIRST_NODE_IP:$SERVER_PORT"
break
fi

CURRENT_TIME=$(date +%s)
if [ $((CURRENT_TIME - START_TIME)) -gt $TIMEOUT ]; then
echo "Error: Server failed to start within $TIMEOUT seconds"
exit 1
fi

echo "Still waiting... ($(($CURRENT_TIME - $START_TIME)) seconds elapsed)"
sleep 60
done

# 정상성 검사
echo "Executing sanity check..."
curl "http://$FIRST_NODE_IP:$SERVER_PORT/v1/completions" \
-H "Content-Type: application/json" \
```

```
-d "{
\"model\": \"default\",
\"prompt\": \"<｜begin_of_sentence｜><｜User｜>hi, how are you?<｜Assistant｜>\"
\"max_tokens\": 2048,
\"temperature\": 0.6
}"
```

2.4 지속적인 헬스 모니터링

```
# 서버 상태 지속 모니터링
while true; do
if ! curl -s -o /dev/null "http://$FIRST_NODE_IP:$SERVER_PORT/health"; then
echo "Error: Server health check failed"
exit 1
fi
sleep 300 # 5분마다 체크
done
```

3. 라우터 시스템 (`slurm/serve_router.slurm`)

3.1 라우터 서버 설정

```
#!/bin/bash
```

```
#SBATCH --job-name=r1-router

#SBATCH --partition=hopper-cpu # CPU 전용 파티션

#SBATCH --qos=high

#SBATCH --nodes=1

#SBATCH --cpus-per-task=8 # 8 CPU 코어

#SBATCH --mem-per-cpu=1875m # 코어당 1.875GB 메모리

#SBATCH --time=30-00:00:00 # 30일 실행

#SBATCH --requeue # 자동 재시작


# 재시작 신호 처리

trap 'scontrol requeue ${SLURM_JOB_ID}; exit 15' SIGUSR1


CONDA_ENV="sglang124"

ROUTER_PORT=39876
```

3.2 라우터 시작 및 관리

```
# 환경 설정

source ~/.bashrc

source "$CONDA_PREFIX/etc/profile.d/conda.sh"

conda activate "$CONDA_ENV" || { echo "Failed to activate conda env $CONDA_EN


# 라우터 시작
```

```
python -m sglang_router.launch_router \
--port "$ROUTER_PORT" \
--host 0.0.0.0 \
--worker-startup-timeout-secs 300

# 라우터 헬스 체크
while true; do
if ! curl -s -o /dev/null "http://localhost:$ROUTER_PORT/health"; then
echo "Error: Router health check failed"
exit 1
fi
sleep 300
done
```

3.3 워커 등록 및 부하 분산

```
# 서버가 시작된 후 라우터에 등록
if [ -n "$ROUTER_ADDRESS" ]; then
echo "Registering with router at $ROUTER_ADDRESS..."
curl -X POST "http://$ROUTER_ADDRESS/add_worker?url=http://$FIRST_NODE_IP:$SE
sleep 10
```

```
fi
```

4. 훈련 클러스터 관리 (`slurm/train.slurm`)

4.1 훈련 작업 설정

```
#!/bin/bash
#SBATCH --job-name=open_r1
#SBATCH --ntasks-per-node=1
#SBATCH --exclusive # 전용 노드 사용
#SBATCH --gres=gpu:8 # 8 GPU 요청
#SBATCH --partition=hopper-prod
#SBATCH --time=3-00:00:00 # 3일 실행

# 클러스터 최적화 설정
module load cuda/12.4
source ~/.bashrc
source openr1/bin/activate

# Weka 파일시스템 캐시 새로고침
echo "Refreshing Weka filesystem..."
find -L /fsx/h4/ -type f | xargs -d '\n' -r -n512 -P64 weka fs tier
```

4.2 명령행 인수 처리

```
# 기본값 설정
MODEL=""
TASK=""
CONFIG_SUFFIX=""
ACCELERATOR=""
DP=1  # 데이터 병렬화
TP=1  # 텐서 병렬화
OPTIONAL_ARGS=""

# 명령행 인수 파싱
while [[ $# -gt 0 ]];  do
case  $1  in
--model)
MODEL="$2"
shift  2
;;
--task)
TASK="$2"  # sft, grpo 등
shift  2
;;
--config)
```

```
CONFIG_SUFFIX="$2"
shift 2
;;
--accelerator)
ACCELERATOR="$2"  # zero2, zero3 등
shift 2
;;
--dp)
DP="$2"
shift 2
;;
--tp)
TP="$2"
shift 2
;;
--args)
OPTIONAL_ARGS="$2"
shift 2
;;
*)
echo "Unknown option: $1"
exit 1
;;
```

```
esac

done
```

4.3 동적 설정 로딩

```
# 설정 파일 경로 구성

CONFIG_FILE=recipes/$MODEL/$TASK/config_$CONFIG_SUFFIX.yaml



# 그래디언트 누적 단계 추출

GRAD_ACC_STEPS=$(grep  'gradient_accumulation_steps'  $CONFIG_FILE | awk  '{prin



# 설정 검증

if [[ -z "$MODEL" || -z "$TASK" || -z "$CONFIG_SUFFIX" || -z "$ACCELERATOR" ]];

echo  "Error: Missing required arguments"

echo  "Usage: sbatch slurm/train.slurm --model MODEL --task TASK --config CONFIG

exit  1

fi
```

5. 실제 배포 시나리오

5.1 단일 노드 서빙

```
# 작은 모델 (7B 이하) 단일 노드 서빙
```

```
sbatch slurm/serve_r1.slurm \
-m "open-r1/OpenR1-Distill-7B" \
-e "openr1"

# 서버 상태 확인
curl http://node-ip:39877/health
curl http://node-ip:39877/v1/models
```

5.2 멀티노드 대형 모델 서빙

```
# 대형 모델 (70B+) 멀티노드 서빙
sbatch --nodes=4 slurm/serve_r1.slurm \
-m "deepseek-ai/DeepSeek-R1" \
-e "sglang124"

# 라우터와 함께 사용
sbatch slurm/serve_router.slurm -e "sglang124"

# 서버를 라우터에 등록
sbatch slurm/serve_r1.slurm \
-m "deepseek-ai/DeepSeek-R1" \
-r "http://router-node:39876"
```

5.3 훈련 작업 실행

```
# SFT 훈련
sbatch slurm/train.slurm \
--model "Qwen2.5-1.5B-Instruct" \
--task "sft" \
--config "demo" \
--accelerator "zero2"

# GRPO 훈련 (vLLM 서버 포함)
sbatch --nodes=2 slurm/train.slurm \
--model "Qwen2.5-7B-Instruct" \
--task "grpo" \
--config "demo" \
--accelerator "zero3" \
--dp 4 --tp 2
```

6. 성능 최적화

6.1 메모리 최적화

```
# 대형 모델용 메모리 최적화 설정
export CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
```

```
export PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:512
export NCCL_P2P_DISABLE=1 # P2P 통신 비활성화 (메모리 절약)

# SGLang 서버 메모리 설정
python -m sglang.launch_server \
--model-path "$MODEL_PATH" \
--tp 8 \
--mem-fraction-static 0.8 \ # GPU 메모리의 80% 사용
--max-running-requests 32 \ # 동시 요청 수 제한
--disable-flashinfer \ # FlashInfer 비활성화 (호환성)
--chunked-prefill-size 4096 # 청크 크기 조정
```

6.2 처리량 최적화

```
# 높은 처리량을 위한 설정
python -m sglang.launch_server \
--model-path "$MODEL_PATH" \
--tp 4 \
--dp 2 \ # 데이터 병렬화 추가
--max-running-requests 128 \ # 더 많은 동시 요청
--max-total-tokens 65536 \ # 토큰 버퍼 증가
--schedule-policy fcfs \ # FCFS 스케줄링
--enable-mixed-chunk # 혼합 청크 처리
```

6.3 지연 시간 최적화

```
# 낮은 지연 시간을 위한 설정
python -m sglang.launch_server \
--model-path "$MODEL_PATH" \
--tp 8 \
--max-running-requests 16 \ # 동시 요청 수 제한
--schedule-policy lpm \ # Longest Prefix Match
--enable-flashinfer \ # FlashInfer 활성화
--attention-reduce-in-fp32 # FP32 어텐션 연산
```

7. 모니터링 및 로깅

7.1 서버 메트릭 수집

```
# 서버 상태 모니터링 스크립트
import requests
import time
import json
from datetime import datetime

def monitor_server(server_url: str, interval: int = 60):
"""서버 상태 지속 모니터링"""
```

```
while True:
try:
# 헬스 체크
health_response = requests.get(f"{server_url}/health", timeout=10)
# 메트릭 수집
metrics_response = requests.get(f"{server_url}/get_server_info", timeout=10)
if health_response.status_code == 200 and metrics_response.status_code == 200:
metrics = metrics_response.json()
# 로그 출력
timestamp = datetime.now().isoformat()
print(f"[{timestamp}] Server Status: OK")
print(f" Running Requests: {metrics.get('num_running_requests', 'N/A')}")
print(f" Waiting Requests: {metrics.get('num_waiting_requests', 'N/A')}")
print(f" GPU Memory Used: {metrics.get('gpu_memory_used', 'N/A')} MB")
print(f" Total Tokens: {metrics.get('total_tokens', 'N/A')}")
# 메트릭을 파일에 저장
with  open("server_metrics.jsonl", "a") as f:
metrics["timestamp"] = timestamp
f.write(json.dumps(metrics) + "\n")
else:
print(f"[{datetime.now().isoformat()}] Server Status: ERROR")
except Exception as e:
print(f"[{datetime.now().isoformat()}] Monitoring Error: {e}")
```

```
time.sleep(interval)


# 사용 예시
if __name__ == "__main__":
monitor_server("http://localhost:39877", interval=30)
```

7.2 로그 분석

```
# 서버 로그 실시간 모니터링
tail -f logs/r1-server_*.out | grep -E "(ERROR|WARNING|Request|GPU)"


# 성능 메트릭 추출
grep "throughput" logs/r1-server_*.out | awk '{print $NF}' | sort -n


# 메모리 사용량 추적
grep "GPU memory" logs/r1-server_*.out | tail -20
```

7.3 알림 시스템

```
# 서버 장애 알림 시스템
import smtplib
from email.mime.text import MIMEText
import requests
```

import time

class ServerAlertSystem:

def __init__(self, server_url: str, email_config: dict):

self.server_url = server_url

self.email_config = email_config

self.last_alert_time = 0

self.alert_cooldown = 300 # 5분 쿨다운

def check_server_health(self) -> bool:

"""서버 상태 확인"""

try:

response = requests.get(f"{self.server_url}/health", timeout=10)

return response.status_code == 200

except:

return False

def send_alert(self, message: str):

"""알림 이메일 발송"""

current_time = time.time()

if current_time - self.last_alert_time < self.alert_cooldown:

return # 쿨다운 중

try:

msg = MIMEText(message)

msg['Subject'] = 'Open R1 Server Alert'

```
msg['From'] = self.email_config['from']

msg['To'] = self.email_config['to']

server = smtplib.SMTP(self.email_config['smtp_server'])

server.send_message(msg)

server.quit()

self.last_alert_time = current_time

print(f"Alert sent: {message}")

except Exception as e:

print(f"Failed to send alert: {e}")

def  monitor(self, check_interval: int = 60):

"""지속적인 모니터링"""

consecutive_failures = 0

while True:

if  self.check_server_health():

consecutive_failures = 0

print(f"[{time.ctime()}] Server is healthy")

else:

consecutive_failures += 1

print(f"[{time.ctime()}] Server health check failed ({consecutive_failures})")

if consecutive_failures >= 3:

self.send_alert(

f"Open R1 server at {self.server_url} has been down for "

f"{consecutive_failures * check_interval} seconds"
```

```
)

time.sleep(check_interval)
```

8. 자동화 및 오케스트레이션

8.1 자동 스케일링

```
# 자동 스케일링 스크립트

import subprocess

import requests

import time

from typing import List


class AutoScaler:

def  __init__(self, router_url: str, min_workers: int = 1, max_workers: int = 4)

self.router_url = router_url

self.min_workers = min_workers

self.max_workers = max_workers

self.active_jobs: List[str] = []

def  get_router_stats(self) -> dict:

"""라우터 통계 가져오기"""

try:

response = requests.get(f"{self.router_url}/stats")
```

```
return response.json()
except:
return {}
def  start_worker(self, model_path: str) -> str:
"""새 워커 시작"""
cmd = [
"sbatch",
"slurm/serve_r1.slurm",
"-m", model_path,
"-r", self.router_url
]
result = subprocess.run(cmd, capture_output=True, text=True)
if result.returncode == 0:
job_id = result.stdout.strip().split()[-1]
self.active_jobs.append(job_id)
print(f"Started worker job: {job_id}")
return job_id
else:
print(f"Failed to start worker: {result.stderr}")
return None
def  stop_worker(self, job_id: str):
"""워커 중지"""
subprocess.run(["scancel", job_id])
```

```
if job_id in self.active_jobs:

self.active_jobs.remove(job_id)

print(f"Stopped worker job: {job_id}")

def  scale_decision(self, stats: dict) -> str:

"""스케일링 결정"""

queue_length = stats.get("queue_length", 0)

active_workers = stats.get("active_workers", 0)

avg_response_time = stats.get("avg_response_time", 0)

# 스케일 업 조건

if (queue_length > 10 or avg_response_time > 30) and active_workers < self.max_w

return  "scale_up"

# 스케일 다운 조건

if queue_length == 0 and avg_response_time < 5 and active_workers > self.min_wor

return  "scale_down"

return  "no_change"

def  run(self, model_path: str, check_interval: int = 300):

"""자동 스케일링 실행"""

while True:

stats = self.get_router_stats()

decision = self.scale_decision(stats)

if decision == "scale_up":

self.start_worker(model_path)

elif decision == "scale_down" and self.active_jobs:
```

```
self.stop_worker(self.active_jobs[-1])
print(f"[{time.ctime()}] Stats: {stats}, Decision: {decision}")
time.sleep(check_interval)
```

8.2 배포 파이프라인

```
#!/bin/bash
# deploy_pipeline.sh - 완전 자동화된 배포 파이프라인

set -e

MODEL_PATH="$1"
DEPLOYMENT_TYPE="$2"  # single, multi, cluster
CONFIG_FILE="$3"

if [[ -z "$MODEL_PATH" || -z "$DEPLOYMENT_TYPE" ]]; then
echo "Usage: $0 MODEL_PATH DEPLOYMENT_TYPE [CONFIG_FILE]"
echo "DEPLOYMENT_TYPE: single, multi, cluster"
exit 1
fi

echo "Starting deployment pipeline for $MODEL_PATH ($DEPLOYMENT_TYPE)"
```

```
# 1. 모델 검증
echo "Validating model..."
python -c "
from transformers import AutoConfig
try:
config = AutoConfig.from_pretrained('$MODEL_PATH')
print(f'Model validated: {config.model_type}')
except Exception as e:
print(f'Model validation failed: {e}')
exit(1)
"

# 2. 리소스 확인
echo "Checking available resources..."
AVAILABLE_NODES=$(sinfo -N -h -t idle | wc -l)
echo "Available nodes: $AVAILABLE_NODES"

# 3. 배포 타입별 실행
case $DEPLOYMENT_TYPE in
"single")
echo "Deploying single node server..."
JOB_ID=$(sbatch --parsable slurm/serve_r1.slurm -m "$MODEL_PATH")
echo "Server job submitted: $JOB_ID"
```

```
;;
"multi")
echo "Deploying multi-node server..."
ROUTER_JOB=$(sbatch --parsable slurm/serve_router.slurm)
echo "Router job submitted: $ROUTER_JOB"
sleep 60 # 라우터 시작 대기
SERVER_JOB=$(sbatch --parsable --nodes=2 slurm/serve_r1.slurm -m "$MODEL_PA
echo "Server job submitted: $SERVER_JOB"
;;
"cluster")
echo "Deploying cluster setup..."
if [[ -z "$CONFIG_FILE" ]]; then
echo "Config file required for cluster deployment"
exit 1
fi
# 설정 파일에서 노드 수 읽기
NODES=$(grep "nodes:" "$CONFIG_FILE" | awk '{print $2}')
sbatch --nodes="$NODES" slurm/serve_r1.slurm -m "$MODEL_PATH"
;;
*)
echo "Unknown deployment type: $DEPLOYMENT_TYPE"
exit 1
;;
```

```
esac

# 4. 배포 상태 확인
echo "Waiting for deployment to be ready..."
sleep 300 # 5분 대기

# 5. 헬스 체크
echo "Performing health check..."
python -c "
import requests
import time
import sys

server_url = 'http://localhost:39877'
max_attempts = 10

for i in range(max_attempts):
try:
response = requests.get(f'{server_url}/health', timeout=10)
if response.status_code == 200:
print('Deployment successful!')
sys.exit(0)
except:
```

```
pass
print(f'Attempt {i+1}/{max_attempts} failed, retrying...')
time.sleep(30)

print('Deployment failed - server not responding')
sys.exit(1)
"

echo "Deployment pipeline completed successfully!"
```

9. 문제 해결 및 디버깅

9.1 일반적인 문제들

메모리 부족 (OOM)

```
# 해결 방법
# 1. 텐서 병렬화 증가
--tp 8 # 더 많은 GPU에 모델 분산

# 2. 메모리 분할 조정
--mem-fraction-static 0.7 # GPU 메모리 사용량 감소

# 3. 동시 요청 수 제한
```

```
--max-running-requests 16  # 동시 요청 수 감소

# 4. 청크 크기 조정
--chunked-prefill-size 2048  # 더 작은 청크 사용
```

네트워크 연결 문제

```
# 네트워크 인터페이스 확인
export GLOO_SOCKET_IFNAME="eth0"  # 올바른 인터페이스 지정
export NCCL_SOCKET_IFNAME="eth0"

# 방화벽 설정 확인
sudo ufw allow 39877  # 서버 포트 열기
sudo ufw allow 45000  # 분산 통신 포트 열기

# DNS 해결 문제
echo "127.0.0.1 $(hostname)" >> /etc/hosts
```

모델 로딩 실패

```
# 모델 경로 확인
ls -la ~/.cache/huggingface/hub/

# 권한 문제 해결
```

```
chmod -R 755 ~/.cache/huggingface/


# 네트워크 문제 시 로컬 캐시 사용
export HF_HUB_OFFLINE=1
```

9.2 성능 디버깅

```
# 성능 프로파일링 스크립트
import requests
import time
import statistics
from concurrent.futures import ThreadPoolExecutor


def benchmark_server(server_url: str, num_requests: int = 100, concurrency: int
"""서버 성능 벤치마크"""
def single_request():
start_time = time.time()
try:
response = requests.post(
f"{server_url}/v1/completions",
json={
"model": "default",
"prompt": "What is the capital of France?",
```

```
"max_tokens": 100,
"temperature": 0.7
},
timeout=60
)
end_time = time.time()
if response.status_code == 200:
return end_time - start_time, len(response.json()["choices"][0]["text"])
else:
return None, 0
except Exception as e:
print(f"Request failed: {e}")
return None, 0
# 동시 요청 실행
with ThreadPoolExecutor(max_workers=concurrency) as executor:
futures = [executor.submit(single_request) for _ in  range(num_requests)]
results = [f.result() for f in futures]
# 성공한 요청만 분석
successful_times = [t for t, _ in results if t is not None]
total_tokens = sum(tokens for _, tokens in results)
if successful_times:
print(f"Benchmark Results:")
print(f" Total Requests: {num_requests}")
```

```
print(f" Successful: {len(successful_times)}")

print(f" Success Rate: {len(successful_times)/num_requests*100:.1f}%")

print(f" Average Latency: {statistics.mean(successful_times):.2f}s")

print(f" Median Latency: {statistics.median(successful_times):.2f}s")

print(f" 95th Percentile: {statistics.quantiles(successful_times, n=20)[18]:.2f}

print(f" Throughput: {len(successful_times)/sum(successful_times):.2f} req/s")

print(f" Tokens/sec: {total_tokens/sum(successful_times):.2f}")

else:

print("All requests failed!")



# 사용 예시

if __name__ == "__main__":

benchmark_server("http://localhost:39877", num_requests=50, concurrency=5)
```

배포 및 서빙 시스템은 Open R1의 실제 운영 환경을 지원하는 핵심 인프라로, 대규모 클러스터에서 안정적이고 효율적인 모델 서빙을 가능하게 합니다. SGLang과 Slurm의 조합으로 확장 가능하고 관리하기 쉬운 배포 솔루션을 제공합니다.